

Chapter 3 Algorithm Evaluation Methods

The goal of predictive modeling is to create models that make good predictions on new data. We don't have access to this new data at the time of training, so we must use statistical methods to estimate the performance of a model on new data. This class of methods is called resampling methods, as they are resampling your available training data. In this tutorial, you will discover how to implement resampling methods from scratch in Perl. After completing this tutorial, you will know:

- How to implement a train and test split of your data.
- How to implement a k-fold cross-validation split of your data.

Let's get started.

3.1 Description

The goal of resampling methods is to make the best use of your training data in order to accurately estimate the performance of a model on new unseen data. Accurate estimates of performance can then be used to help you choose which set of model parameters to use or which model to select. Once you have chosen a model, you can train for final model on the entire training dataset and start using it to make predictions. There are two common resampling methods that you can use:

- A train and test split of your data.
- k-fold cross-validation.

In this tutorial, we will look at using each and when to use one method over the other.

3.2 Tutorial

This tutorial is divided into 3 parts:

1. Train and Test Split.
2. k-fold Cross-Validation Split.
3. How to Choose a Resampling Method.

These steps will provide the foundations you need to handle resampling your dataset to estimate algorithm performance on new data.

3.2.1 Train and Test Split

The train and test split is the easiest resampling method. As such, it is the most widely used. The train and test split involves separating a dataset into two parts:

1. Training Dataset.
2. Test Dataset.

The training dataset is used by the machine learning algorithm to train the model. The test dataset is held back and is used to evaluate the performance of the model. The rows assigned to each dataset are randomly selected. This is an attempt to ensure that the training and evaluation of a model is objective. If

multiple algorithms are compared or multiple configurations of the same algorithm are compared, the same train and test split of the dataset should be used. This is to ensure that the comparison of performance is consistent or apples-to-apples. We can achieve this by seeding the random number generator the same way before splitting the data, or by holding the same split of the dataset for use by multiple algorithms. We can implement the train and test split of a dataset in a single function. Below is a function named `train_test_split()` to split a dataset into a train and test split. It accepts two arguments: the dataset to split as a list of lists and an optional split percentage. A default split percentage of 0.6 or 60% is used. This will assign 60% of the dataset to the training dataset and leave the remaining 40% to the test dataset. A 60/40 for train/test is a good default split of the data. The function first calculates how many rows the training set requires from the provided dataset. A copy of the original dataset is made. Random rows are selected and removed from the copied dataset and added to the train dataset until the train dataset contains the target number of rows. The rows that remain in the copy of the dataset are then returned as the test dataset. The `int(rand())` function from the random model is used to generate a random integer in the range between 0 and the size of the list.

```
In [1]: use strict;
        use warnings;
        use Data::Dump qw(dump);
        use List::Util qw(shuffle);
        use sml;

In [2]: # Defined in Section 3.2.1 Train and Test Split
        # Function To Split a Dataset.
        # Split a dataset into a train and test set
        sub train_test_split{
            my ($self, $dataset, %args) = (splice (@_, 0, 2), split=>0.6, @_);

            my $train_size = int($args{split} * @$dataset);
            my @idx        = shuffle (0 .. $#$dataset);
            my @train_idx  = @idx[0 .. $train_size -1];
            my @test_idx   = @idx[$train_size .. $#$dataset];
            my @train      = @$dataset[@train_idx];
            my @test       = @$dataset[@test_idx];

            return \@train, \@test;
        }

        sml->add_to_class('train_test_split', \&{'train_test_split'});
```

```
Out[2]: *sml::train_test_split
```

We can test this function using a contrived dataset of 10 rows, each with a single column. The complete example is listed below.

The example fixes the random seed before splitting the training dataset. This is to ensure the exact same split of the data is made every time the code is executed. This is handy if we want to use the same split many times to evaluate and compare the performance of different algorithms. Running the example produces the output below. The data in the train and test set is printed, showing that 6/10 or 60% of the records were assigned to the training dataset and 4/10 or 40% of the records were assigned to the test set.

```
In [3]: # Example of Splitting a Contrived Dataset into Train and Test

        # test train/test split
        srand(1);
```

```

my $dataset      = [[1], [2], [3], [4], [5], [6], [7], [8], [9], [10]];
my ($train, $test) = sml->train_test_split($dataset);
printf "%s\n", dump ($train);
printf "%s\n", dump ($test);
# Example Output from Splitting a Dataset.
# [[6], [2], [8], [9], [10], [4]]
# [[3], [7], [5], [1]]

```

```

[[6], [2], [8], [9], [10], [4]]
[[3], [7], [5], [1]]

```

Out[3]: 1

3.2.2 k-fold Cross-Validation Split

A limitation of using the train and test split method is that you get a noisy estimate of algorithm performance. The k-fold cross-validation method (also called just cross-validation) is a resampling method that provides a more accurate estimate of algorithm performance. It does this by first splitting the data into k groups. The algorithm is then trained and evaluated k times and the performance summarized by taking the mean performance score. Each group of data is called a fold, hence the name k-fold cross-validation. It works by first training the algorithm on the k-1 groups of the data and evaluating it on the kth hold-out group as the test set. This is repeated so that each of the k groups is given an opportunity to be held out and used as the test set. As such, the value of k should be divisible by the number of rows in your training dataset, to ensure each of the k groups has the same number of rows. You should choose a value for k that splits the data into groups with enough rows that each group is still representative of the original dataset. A good default to use is k=3 for a small dataset or k=10 for a larger dataset. A quick way to check if the fold sizes are representative is to calculate summary statistics such as mean and standard deviation and see how much the values differ from the same statistics on the whole dataset. We can reuse what we learned in the previous section in creating a train and test split here in implementing k-fold cross-validation. Instead of two groups, we must return k-folds or k groups of data. Below is a function named cross validation split() that implements the cross-validation split of data. As before, we create a copy of the dataset from which to draw randomly chosen rows. We calculate the size of each fold as the size of the dataset divided by the number of folds required.

$$\text{fold size} = \text{count}(\text{rows}) / \text{count}(\text{folds})$$

(3.1)

If the dataset does not cleanly divide by the number of folds, there may be some remainder rows and they will not be used in the split. We then create a list of rows with the required size and add them to a list of folds which is then returned at the end.

```

In [4]: # Defined in Section 3.2.2 k-fold Cross-Validation Split
# Function Create A Cross-Validation Split.
# Split a dataset into $ k $ folds
sub cross_validation_split{
    my ($self, $dataset, %args) = (splice (@_, 0, 2), n_folds=>10, @_);

    my @dataset_split;
    my $fold_size = int(@$dataset / $args{n_folds});
    my @idx       = shuffle (0 .. $#dataset);
    for my $i (0 .. $args{n_folds} - 1){
        my @fold_idx = @idx[$i * $fold_size .. ($i + 1) * $fold_size - 1];
        push @dataset_split, [@$dataset[@fold_idx]];
    }

    return \@dataset_split;
}

```

```
}
sml->add_to_class('cross_validation_split', \&{'cross_validation_split'});
```

Out[4]: *sml::cross_validation_split

We can test this resampling method on the same small contrived dataset as above. Each row has only a single column value, but we can imagine how this might scale to a standard machine learning dataset.

The complete example is listed below. As before, we fix the seed for the random number generator to ensure that each time the code is executed that the same rows are used in the same folds. A k value of 4 is used for demonstration purposes. We would expect that the 10 rows divided into 4 folds will result in 2 rows per fold, with a remainder of 2 that will not be used in the split.

```
In [5]: # Example of a Cross-Validation Split of a Contrived Dataset.
# test cross validation split
srand(1);
my $dataset = [[1], [2], [3], [4], [5], [6], [7], [8], [9], [10]];
my $folds = sml->cross_validation_split($dataset, n_folds=>5);
printf "%s\n", dump ($folds);
# Example Output from Creating a Cross-Validation Split.
# [[[6], [2]], [[8], [9]], [[10], [4]], [[3], [7]], [[5], [1]]]
```

```
[[[6], [2]], [[8], [9]], [[10], [4]], [[3], [7]], [[5], [1]]]
```

Out[5]: 1

```
In [6]: my ($dataset, $header) = sml->load_csv('../data/iris.csv');
printf "rows: %d\n", scalar @$dataset;
printf "cols: %d\n", scalar @{$dataset->[0]};
print dump @$dataset[0 .. 4];
# rows: 150
# cols: 5
# (
#   [5.1, 3.5, 1.4, 0.2, "Iris-setosa"],
#   [4.9, "3.0", 1.4, 0.2, "Iris-setosa"],
#   [4.7, 3.2, 1.3, 0.2, "Iris-setosa"],
#   [4.6, 3.1, 1.5, 0.2, "Iris-setosa"],
#   ["5.0", 3.6, 1.4, 0.2, "Iris-setosa"],
# )
```

rows: 150

cols: 5

```
(
  [5.1, 3.5, 1.4, 0.2, "Iris-setosa"],
  [4.9, "3.0", 1.4, 0.2, "Iris-setosa"],
  [4.7, 3.2, 1.3, 0.2, "Iris-setosa"],
  [4.6, 3.1, 1.5, 0.2, "Iris-setosa"],
  ["5.0", 3.6, 1.4, 0.2, "Iris-setosa"],
)
```

Out[6]: 1

```
In [7]: my ($lookup, $rlookup) = sml->str_column_to_int($dataset, -1);
printf "lookup:\n%s\n", dump $lookup;
printf "rlookup:\n%s\n", dump $rlookup;
printf "Modified dataset:\n%s", dump @$dataset[0 .. 4];
```

```

lookup:
{ "Iris-setosa" => 0, "Iris-versicolor" => 1, "Iris-virginica" => 2 }
rlookup:
{ "0" => "Iris-setosa", "1" => "Iris-versicolor", "2" => "Iris-virginica" }
Modified dataset:
(
  [5.1, 3.5, 1.4, 0.2, 0],
  [4.9, "3.0", 1.4, 0.2, 0],
  [4.7, 3.2, 1.3, 0.2, 0],
  [4.6, 3.1, 1.5, 0.2, 0],
  ["5.0", 3.6, 1.4, 0.2, 0],
)

```

Out[7]: 1

```

In [8]: sub count_labels{
  my ($self, $dataset) = @_;
  my %counts = ();
  map {$counts{"$_->[-1]"}++} @$dataset;
  return \%counts;
}

sml->add_to_class('count_labels', \&{'count_labels'});

```

Out[8]: *sml::count_labels

```

In [9]: my $counts = sml->count_labels($dataset);
  print dump $counts;
  # { "Iris-setosa" => 50, "Iris-versicolor" => 50, "Iris-virginica" => 50 }

{ "0" => 50, "1" => 50, "2" => 50 }

```

Out[9]: 1

```

In [10]: for my $key (keys %$counts){
  printf "%s => %d ", $rlookup->{$key}, $counts->{$key};
}

Iris-virginica => 50 Iris-setosa => 50 Iris-versicolor => 50

```

```

In [11]: srand(1);
  my ($train, $test) = sml->train_test_split($dataset, split=>0.8);
  printf "train size:%d, test size:%d", scalar (@$train),
        scalar (@$test);
  # train size:120, test size:30

train size:120, test size:30

```

Out[11]: 1

```

In [12]: print dump (sml->count_labels($train));
  # { "Iris-setosa" => 40, "Iris-versicolor" => 42, "Iris-virginica" => 38 }

{ "0" => 40, "1" => 42, "2" => 38 }

```

Out[12]: 1

```

In [13]: print dump (sml->count_labels($test));
  # { "Iris-setosa" => 10, "Iris-versicolor" => 8, "Iris-virginica" => 12 }

{ "0" => 10, "1" => 8, "2" => 12 }

```

Out[13]: 1

```

In [14]: print dump @$train[0 .. 9];

```

```
(
  [7.2, 3.2, "6.0", 1.8, 2],
  [5.9, "3.0", 4.2, 1.5, 1],
  [6.5, "3.0", 5.5, 1.8, 2],
  [5.5, 2.4, 3.7, "1.0", 1],
  [5.8, 2.6, "4.0", 1.2, 1],
  [4.8, 3.4, 1.6, 0.2, 0],
  [4.8, 3.4, 1.9, 0.2, 0],
  [6.1, 2.9, 4.7, 1.4, 1],
  [6.4, 3.2, 5.3, 2.3, 2],
  ["5.0", 3.2, 1.2, 0.2, 0],
)
```

Out[14]: 1

In [15]: `print dump @$test[0 .. 9];`

```
(
  [6.7, 2.5, 5.8, 1.8, 2],
  [4.6, 3.2, 1.4, 0.2, 0],
  [6.1, 2.8, 4.7, 1.2, 1],
  [6.5, 3.2, 5.1, "2.0", 2],
  [4.4, 3.2, 1.3, 0.2, 0],
  [7.7, 3.8, 6.7, 2.2, 2],
  [6.3, 2.5, 4.9, 1.5, 1],
  [6.7, 3.3, 5.7, 2.5, 2],
  [6.8, 3.2, 5.9, 2.3, 2],
  [5.5, 2.4, 3.8, 1.1, 1],
)
```

Out[15]: 1

In [16]: `srand(1);`
`my $folds = sml->cross_validation_split($dataset, n_folds=>10);`
`printf "%s\n", dump (@$folds[0 .. 1]);`

```
(
  [
    [7.2, 3.2, "6.0", 1.8, 2],
    [5.9, "3.0", 4.2, 1.5, 1],
    [6.5, "3.0", 5.5, 1.8, 2],
    [5.5, 2.4, 3.7, "1.0", 1],
    [5.8, 2.6, "4.0", 1.2, 1],
    [4.8, 3.4, 1.6, 0.2, 0],
    [4.8, 3.4, 1.9, 0.2, 0],
    [6.1, 2.9, 4.7, 1.4, 1],
    [6.4, 3.2, 5.3, 2.3, 2],
    ["5.0", 3.2, 1.2, 0.2, 0],
    [7.1, "3.0", 5.9, 2.1, 2],
    [6.9, 3.1, 4.9, 1.5, 1],
    [6.1, "3.0", 4.9, 1.8, 2],
    [5.2, 2.7, 3.9, 1.4, 1],
    [6.3, 2.9, 5.6, 1.8, 2],
  ],
  [
    [6.2, 2.8, 4.8, 1.8, 2],
    [6.7, "3.0", 5.2, 2.3, 2],
    [5.4, 3.7, 1.5, 0.2, 0],
    [5.7, "3.0", 4.2, 1.2, 1],
    [6.5, "3.0", 5.2, "2.0", 2],
    [4.5, 2.3, 1.3, 0.3, 0],
    [5.6, "3.0", 4.1, 1.3, 1],
    [5.4, 3.9, 1.3, 0.4, 0],
    [5.1, 3.7, 1.5, 0.4, 0],
    [4.9, 3.1, 1.5, 0.1, 0],
    [5.6, 2.9, 3.6, 1.3, 1],
    [5.6, 2.8, 4.9, "2.0", 2],
    [6.9, 3.1, 5.4, 2.1, 2],
    [5.8, 2.8, 5.1, 2.4, 2],
    [6.3, 3.4, 5.6, 2.4, 2],
  ],
)
```

Out[16]: 1

```
In [17]: printf "rows per fold:%d\n", scalar @{$folds->[0]}
```

rows per fold:15

Out[17]: 1

```
In [18]: for my $medicion (map {sml->count_labels($_)} @{$folds}){
  printf "%s\n", dump $medicion;
}
```

```
{ "0" => 3, "1" => 6, "2" => 6 }
{ "0" => 5, "1" => 3, "2" => 7 }
{ "0" => 5, "1" => 4, "2" => 6 }
{ "0" => 5, "1" => 4, "2" => 6 }
{ "0" => 6, "1" => 8, "2" => 1 }
{ "0" => 5, "1" => 8, "2" => 2 }
{ "0" => 6, "1" => 4, "2" => 5 }
{ "0" => 5, "1" => 5, "2" => 5 }
{ "0" => 4, "1" => 4, "2" => 7 }
{ "0" => 6, "1" => 4, "2" => 5 }
```

3.2.3 How to Choose a Resampling Method

The gold standard for estimating the performance of machine learning algorithms on new data is k-fold cross-validation. When well-configured, k-fold cross-validation gives a robust estimate of performance compared to other methods such as the train and test split. The downside of cross-validation is that it can be time-consuming to run, requiring k different models to be trained and evaluated. This is a problem if you have a very large dataset or if you are evaluating a model that takes a long time to train. The train and test split resampling method is the most widely used. This is because it is easy to understand and implement, and because it gives a quick estimate of algorithm performance. Only a single model is constructed and evaluated. Although the train and test split method can give a noisy or unreliable estimate of the performance of a model on new data, this becomes less of a problem if you have a very large dataset. Large datasets are those in the hundreds of thousands or millions of records, large enough that splitting it in half results in two datasets that have nearly equivalent statistical properties. In such cases, there may be little need to use k-fold cross-validation as an evaluation of the algorithm and a train and test split may be just as reliable.

3.3 Extensions

In this tutorial, we have looked at the two most common resampling methods. There are other methods you may want to investigate and implement as extensions to this tutorial. For example:

- Repeated Train and Test. This is where the train and test split is used, but the process is repeated many times.
- LOOCV or Leave One Out Cross-Validation. This is a form of k-fold cross-validation where the value of k is fixed at 1.
- Stratification. In classification problems, this is where the balance of class values in each group is forced to match the original dataset.

3.4 Review

In this tutorial, you discovered how to implement resampling methods in Python from scratch. Specifically, you learned:

- How to implement the train and test split method.
- How to implement the k-fold cross-validation method.
- When to use each method.